Python for Artificial Intelligence

NLP Foundations: Cleaning, Tokens, Tags and Entities

Week 5, Day 4

OVERVIEW

Today's Plan

- Today is about the toolbox that sits underneath every language project: how a program reads, cleans and breaks up text
- Hour 1: what NLP is, where it is used, and the pipeline every project follows. Then we clean messy text
- Hour 2: tokenization, stop words, stemming, part-of-speech tags and lemmatization
- Hour 3: named entities, n-grams and the first look at turning words into numbers
- Our running data is the Bitext customer-support dataset from Kaggle: 26,872 short customer messages. Day 24 uses everything from today to build a support-desk bot

Hour 1

What Is NLP?

WHY, WHAT, LIMITS

Before We Start: NLP

Why it matters, what it is, and where it stops working.

	NLP
Why it matters	Most of the world's information is written or spoken language: reviews, chats, emails, voice notes. A program that cannot read language cannot use any of it
What it is	NLP is the part of AI that lets programs read, understand and produce human language, in text or speech
Limitations	Language is ambiguous, changes quickly and differs between regions. Models still make mistakes on slang, sarcasm, typos and mixed languages such as Roman Urdu

CONCEPT

Natural Language Processing

- NLP is the part of AI that helps machines understand and work with human language, in text or in speech
- It sits where three fields meet: linguistics (how language works), machine learning (learning from examples) and programming
- It powers speech recognition, translation, summaries and the chat assistants you already use
- Language is hard for a computer: the same idea has many wordings, words have several meanings, and people make typos and mix languages

CONCEPT

Two Halves: NLU and NLG

- Natural Language Understanding (NLU): turn language into something a program can act on. Example: read "where is my parcel" and decide the intent is track_order
- Natural Language Generation (NLG): turn data into language that reads like a person wrote it. Example: "Your parcel left Lahore today and arrives Friday"
- A support bot needs both. Day 24 builds the NLU half with a classifier. LLMs do both halves in one model

CONCEPT

Where You Already Meet NLP

Each of these is NLP doing a job.

Application	What it does
Voice assistants	Turn speech to text, then work out what you asked
Customer-support chatbots	Find the intent of a message and reply
Spam filtering	Classify an email as spam or not
Autocorrect and predictive typing	Guess the next word or fix a typo
Translation	Convert text from one language to another
Sentiment tracking	Measure whether posts about a brand are positive or negative
Search and recommendation	Find documents that mean the same as the query

CONCEPT

Common NLP Tasks

Most real products combine several of these.

Task	What it does	Support-desk example
Text classification	Give a text a label	Message -> get_refund
Sentiment analysis	Positive, negative or neutral	Is the customer angry?
Named entity recognition	Find names, places, dates, amounts	Pull out the order number
Machine translation	Text from one language to another	Roman Urdu -> English
Summarization	Shorter text, same meaning	Summarize a 40-message thread
Question answering	Read text, answer a question	What is the refund window?

REFERENCE

The Python NLP Libraries

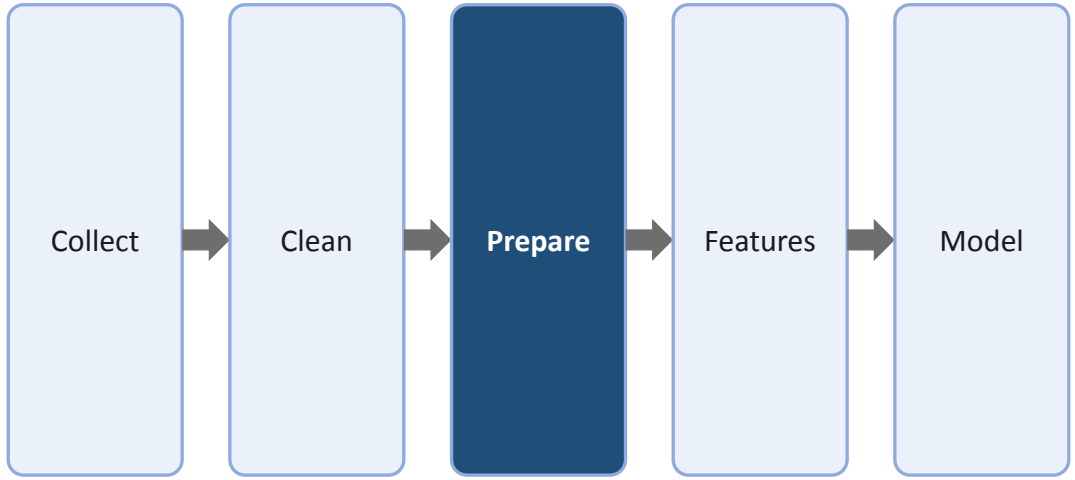
We use the first three today; scikit-learn comes back in the vectorizing part.

Library	Level	Best for
re (built in)	Beginner	Cleaning text with patterns
NLTK	Beginner	Learning: tokenizing, stemming, lemmatizing, POS tags
spaCy	Intermediate	Fast, ready-made pipeline with named entities
TextBlob	Beginner	Quick sentiment and spelling correction
scikit-learn	Beginner	CountVectorizer, TF-IDF and classifiers
Gensim	Intermediate	Topic models and word vectors
Transformers (Hugging Face)	Advanced	Pretrained deep-learning language models

CONCEPT

The Typical NLP Pipeline

- Collect: gather the text (CSV files, scraped pages, logs, transcripts)
- Clean: remove noise such as links, HTML and stray symbols
- Preprocess: tokens, stop words, lemmas, tags
- Features: turn the tokens into numbers
- Model: train a classifier or use a pretrained model. Then evaluate it, deploy it, and keep monitoring it, because language changes



CONCEPT

Kinds of NLP Data

- Raw text: books, articles, social media, reviews, emails. Unstructured: no labels
- Labelled text: each text carries a tag for supervised learning. Sentiment labels, topic labels, and in our data an intent label
- Annotated text: words tagged with extra information such as part of speech or entity type
- Conversations: question and answer pairs, chat logs. Speech: audio plus a transcript
- Our Bitext data is labelled text: a customer message plus its intent, and a prepared reply

EXAMPLE

Meet Our Data

On Kaggle, click Add Input and search for "Bitext customer support". Copy the CSV's path from the file listing (Day 24 shows how) and use it in read_csv.

```
import pandas as pd

df = pd.read_csv('bitext.csv')
print(df.shape)
print(df['instruction'].sample(3, random_state=2).str[:45].to_string())
print(df['instruction'].str.split().str.len().mean().round(1))
# >> (26872, 5)
#    12487    can i place a damn order from {{Delivery Coun
#    3035     I do not know how I can see the cancellation
#    11704       I want help canceling the standard account
#    8.7
```

Hour 1, continued

Cleaning Text

WHY, WHAT, LIMITS

Before We Start: Text Cleaning

Why it matters, what it is, and where it stops working.

	Text Cleaning
Why it matters	'Refund', 'refund' and 'REFUND!!' are three different strings to a computer. Cleaning makes the same idea look the same, so a model learns from it instead of from noise
What it is	Cleaning is a chain of small steps that remove or standardise noise: case, links, HTML, digits, symbols, repeated letters, extra spaces
Limitations	Every step also removes something. There is no single correct recipe, and an aggressive step can delete meaning, such as an email address or all Urdu text

CONCEPT

Why Clean Text at All?

- Imagine you scrape product reviews and get: "OMG!!! This phone is AMAZING 😍😍!!! Visit www.shop.com now!!!"
- To a computer, "AMAZING", "amazing" and "Amazing" are three different words. The link, the emoji and the shouting add noise but no meaning
- Cleaning removes the noise so the same idea always looks the same. Each step is a small function; we chain them
- There is no single correct recipe. The right cleaning depends on the task, so we always check what each step does to real text

Regex

Patterns for Finding and Changing Text

WHY, WHAT, LIMITS

Before We Start: Regular Expressions (regex)

Why it matters, what it is, and where it stops working.

	Regular Expressions (regex)
Why it matters	Almost every cleaning step and every rule-based extractor is a pattern: remove links, find order numbers, spot emails. One short pattern replaces dozens of lines of loops and if statements
What it is	A regular expression is a small pattern language. You describe the shape of the text you want, and the engine finds, extracts or replaces every piece that fits. Python's re module runs it
Limitations	Regex matches shape, not meaning. It cannot tell a refund request from a joke, nests badly, is hard to read, and a careless pattern can be slow or wrong on text you did not test

REGEX RULES

Rules 1: Literals, Dot and Escaping

The simplest patterns. Special characters must be escaped with a backslash to mean themselves.

Pattern	Meaning	Example
abc	plain characters match themselves, in order	cat finds cat in concat
.	any one character except a newline	c.t matches cat and c9t
\. \? \+ \(	a backslash makes a special character literal	\.pk matches .pk
\\	a literal backslash	C:\\data matches C:\data
. ^ $ * + ? { } [ ] \ | ( )	the special characters: escape them to match them	re.escape() does it for you
\n \t	a newline; a tab	\n splits lines
\uXXXX \UXXXXXXXX	a character by its Unicode code	\U0001F600 is a smiley
r'...'	raw string: Python keeps every backslash as written	r'\d' not '\\d'

WHY, WHAT, LIMITS

Before We Start: Character Classes

Why it matters, what it is, and where it stops working.

	Character Classes
Why it matters	A message holds many different digits, letters and spaces. Classes let one pattern say 'any digit' or 'any letter' instead of listing them
What it is	A class matches ONE character out of a set. Shorthand classes (\d, \w, \s) cover the common sets; square brackets let you define your own
Limitations	A class matches a single character only; quantifiers extend it. [a-z] covers English letters only, so Urdu is missed, while \w in Python 3 does include Unicode letters

REGEX RULES

Rules 2: Shorthand Classes

Each one has an upper-case opposite.

Pattern	Meaning	Example
\d	a digit (0-9, and other Unicode digits)	\d+ finds 48213
\D	anything that is not a digit	\D+ finds 'Order #'
\w	a letter, digit or underscore (Unicode letters included)	\w+ finds a word
\W	anything that is not \w	\W finds # , - .
\s	a space, tab, newline or similar	\s+ finds a run of spaces
\S	anything that is not whitespace	\S+ finds a chunk with no spaces

REGEX RULES

Rules 3: Your Own Sets

Square brackets match one character from the set.

Pattern	Meaning	Example
[abc]	one of a, b or c	[aeiou] finds a vowel
[a-z] [A-Z] [0-9]	a range of characters	[A-Z]\w+ finds Order, Ali
[a-zA-Z0-9_]	ranges combined in one set	same as \w for English
[^abc]	one character NOT in the set (^ first)	[^a-z\s] finds symbols and digits
[.] [?] [$]	inside a set most special characters are literal	[.] matches a dot only
[\d\s]	shorthand classes work inside a set	a digit or a space
[-+] [a-z-]	a hyphen first or last is a literal hyphen	[+-]?\d+ finds -5 or +5

WHY, WHAT, LIMITS

Before We Start: Quantifiers

Why it matters, what it is, and where it stops working.

	Quantifiers
Why it matters	Words, numbers and phone numbers have different lengths. Quantifiers say how many times to repeat, so one pattern handles 4 digits or 12
What it is	A quantifier follows a character or group and sets how many times it can repeat: zero or one, one or more, an exact number or a range
Limitations	By default quantifiers are greedy: they take as much text as they can, which often swallows too much. They also count characters, not meaning, so \d{5} matches the first five digits of any longer number

REGEX RULES

Rules 4: Quantifiers

Placed straight after what they repeat.

Pattern	Meaning	Example
?	zero or one (optional)	colou?r matches color and colour
*	zero or more	ab* matches a, ab, abbb
+	one or more	\d+ matches 7 or 1234567
{n}	exactly n times	\d{4} matches 0300
{n,}	n or more times	\d{5,} matches a long number
{n,m}	between n and m times	\d{2,4} matches 12, 123, 1234
{,m}	up to m times (same as {0,m})	\d{,3}

WHY, WHAT, LIMITS

Before We Start: Greedy and Lazy Matching

Why it matters, what it is, and where it stops working.

	Greedy and Lazy Matching
Why it matters	The most common regex bug is a pattern that matches far too much. The usual case is removing HTML tags: the greedy pattern removes the text between them too
What it is	Greedy quantifiers take the longest possible match. Adding a ? after the quantifier makes it lazy: it takes the shortest match that still lets the whole pattern succeed
Limitations	Lazy is not a cure-all: it still fails on nested structures. Often the safest pattern is the explicit one, such as [^>]* for 'anything except the closing bracket'

REGEX RULES

Rules 5: Greedy versus Lazy

Same pattern, one extra character, very different result.

Pattern	Meaning	Example
* + ? {n,m}	greedy: as much as possible	<.*> on 'x' matches the whole string
*? +? ?? {n,m}?	lazy: as little as possible	<.*?> matches , then
[^>]*	anything except one character (explicit)	<[^>]*> matches and
Rule of thumb	prefer the explicit set, then lazy, then greedy	test on text with two matches on one line

EXAMPLE

Classes, Quantifiers and Greedy Matching in Code

The first block uses classes and quantifiers. The second shows greedy against lazy on the same text.

```
import re

text = "Order #48213 for Ali, call 0300-1234567. Late again!!"

print(re.findall(r'\d+', text))            # digits, one or more
print(re.findall(r'\w+', text))            # word characters
print(re.findall(r'[A-Z]\w+', text))       # capital letter then letters
print(re.findall(r'[^\w\s]', text))        # NOT a letter, digit or space
print(re.findall(r'\d{4}-\d{7}', text))    # exactly 4 digits, dash, 7 digits
print(re.findall(r'o?rder', 'order rder'))  # the o is optional
# >> ['48213', '0300', '1234567']
#    ['Order', '48213', 'for', 'Ali', 'call', '0300', '1234567', 'Late', 'again']
#    ['Order', 'Ali', 'Late']
#    ['#', ',', '-', '.', '!', '!']
#    ['0300-1234567']
#    ['order', 'rder']
```

EXAMPLE

Greedy, Lazy and Explicit

Run all three on the same text and compare.

```
import re

html = "<b>refund</b> and <i>order</i>"

print(re.findall(r'<.*>', html))     # greedy: takes as much as it can
print(re.findall(r'<.*?>', html))    # lazy: stops at the first >
print(re.findall(r'<[^>]*>', html))  # safer: anything except >
# >> ['<b>refund</b> and <i>order</i>']
#    ['<b>', '</b>', '<i>', '</i>']
#    ['<b>', '</b>', '<i>', '</i>']
```

WHY, WHAT, LIMITS

Before We Start: Anchors and Word Boundaries

Why it matters, what it is, and where it stops working.

	Anchors and Word Boundaries
Why it matters	Without anchors a pattern matches anywhere. 'cat' also matches inside 'concat', and 'order' matches the end of 'reorder'. Anchors pin the match to a position
What it is	Anchors match a position, not a character: the start or end of the text, the start or end of a line, or the edge of a word
Limitations	^ and $ mean the whole text unless you turn on multiline mode. \b depends on what counts as a word character, so it behaves oddly around hyphens, apostrophes and some scripts

REGEX RULES

Rules 6: Anchors and Boundaries

They take up no characters. They only check where you are.

Pattern	Meaning	Example
^	start of the text (start of each line with re.MULTILINE)	^refund finds a leading refund
$	end of the text (or each line with re.MULTILINE)	order$ finds a trailing order
\A	start of the text only, never affected by flags	\Aorder
\Z	end of the text only, never affected by flags	order\Z
\b	a word boundary: between \w and \W	\bcat\b is the word cat only
\B	not a word boundary	\Bcat finds cat inside concat

EXAMPLE

Anchors and Boundaries in Code

Compare the patterns with and without the multiline flag, and with and without \b.

```
import re

msg = "cancel my order\nrefund my order"

print(re.findall(r'^\w+', msg))                    # start of the whole text
print(re.findall(r'^\w+', msg, re.MULTILINE))     # start of every line
print(re.findall(r'order$', msg))                  # end of the text
print(re.findall(r'\bcat\b', 'cat concat cat.')) # whole word only
print(re.findall(r'cat', 'cat concat cat.'))      # no boundary: matches inside
# >> ['cancel']
#    ['cancel', 'refund']
#    ['order']
#    ['cat', 'cat']
#    ['cat', 'cat', 'cat']
```

WHY, WHAT, LIMITS

Before We Start: Groups and Alternation

Why it matters, what it is, and where it stops working.

	Groups and Alternation
Why it matters	Real messages vary: 'cancel order' or 'close order'. Groups let you choose between options, pull out pieces of a match, and reuse a piece later in the pattern or in a replacement
What it is	Parentheses group part of a pattern. A capturing group remembers what it matched; a non-capturing group only groups. The pipe | means or. A backreference such as \1 matches the same text again
Limitations	Many groups make patterns hard to read, and numbered groups break when you add one. Name them. Alternation tries options left to right, so put the longer option first (cat|category would stop at cat)

REGEX RULES

Rules 7: Groups, Alternation and Backreferences

Parentheses do two jobs: grouping and capturing.

Pattern	Meaning	Example
( )	capturing group: group and remember the match	(\d{4})-(\d{7}) gives two parts
(?:)	non-capturing group: group only	(?:cancel|close) order
(?P<name>)	named capturing group	(?P<code>\d{4})
a|b	either a or b, tried left to right	cancel|close|delete
\1 \2	the same text again, by group number	(\w)\1 finds a doubled letter
(?P=name)	the same text again, by group name	(?P<c>\w)(?P=c)
\1 in sub()	a captured group inside a replacement	swap two words: r'\2 \1'

EXAMPLE

Groups in Code

group() returns what a group matched. groupdict() returns the named groups. sub() can reorder the groups.

```
import re

m = re.search(r'(\d{4})-(\d{7})', 'call 0300-1234567 now')
print(m.group(0), m.group(1), m.group(2))

m = re.search(r'(?P<code>\d{4})-(?P<num>\d{7})', 'call 0300-1234567')
print(m.group('code'), m.groupdict())

print(re.findall(r'(?:cancel|close) (?:my )?order', 'cancel my order, close order'))
print(re.findall(r'(\w)\1', 'refund coffee ball'))     # a doubled letter
print(re.sub(r'(\w+) (\w+)', r'\2 \1', 'refund order'))  # swap two words
# >> 0300-1234567 0300 1234567
#    0300 {'code': '0300', 'num': '1234567'}
#    ['cancel my order', 'close order']
#    ['f', 'e', 'l']
#    order refund
```

WHY, WHAT, LIMITS

Before We Start: Lookaround

Why it matters, what it is, and where it stops working.

	Lookaround
Why it matters	Sometimes the text you want depends on what is next to it, and you want only the number, not the 'Rs ' in front. Lookaround checks the neighbours without including them in the match
What it is	A lookaround is a zero-width test. Lookahead checks what follows, lookbehind what comes before. Each can be positive (must be there) or negative (must not be there)
Limitations	A lookbehind in Python must have a fixed length, so (?<=Rs) works but (?<=\w+) does not. Lookaround makes patterns harder to read: use it only when groups are not enough

REGEX RULES

Rules 8: Lookaround

Zero-width: the checked text is not part of the match.

Pattern	Meaning	Example
(?=...)	positive lookahead: followed by ...	\d+(?= USD) finds 30 in '30 USD'
(?!...)	negative lookahead: NOT followed by ...	\d+(?! USD) skips amounts in USD
(?<=...)	positive lookbehind: preceded by ...	(?<=Rs)\d+ finds 500 in 'Rs 500'
(?<!...)	negative lookbehind: NOT preceded by ...	(?<!\w)@\w+ skips the @ inside an email

EXAMPLE

Lookaround in Code

Same text, four checks. Only the number is returned, never the words around it.

```
import re

t = "price Rs 500, Rs 1200, 30 USD"

print(re.findall(r'\d+(?= USD)', t))       # digits followed by ' USD'
print(re.findall(r'\d+(?!\d| USD)', t))    # digits NOT followed by ' USD'
print(re.findall(r'(?<=Rs )\d+', t))       # digits preceded by 'Rs '
print(re.findall(r'(?<!Rs )\b\d+', t))     # digits NOT preceded by 'Rs '
# >> ['30']
#    ['500', '1200']
#    ['500', '1200']
#    ['30']
```

WHY, WHAT, LIMITS

Before We Start: Flags

Why it matters, what it is, and where it stops working.

	Flags
Why it matters	Customers write REFUND, Refund and refund, and messages can span several lines. Flags change how the whole pattern behaves, so you do not repeat yourself
What it is	A flag is a switch passed as a third argument to re functions, or written inside the pattern as (?i). Combine several with the | operator
Limitations	A flag applies to the whole pattern. Case-insensitive matching depends on the language and can behave unexpectedly for some Unicode letters, so test it on your own text

REGEX RULES

Rules 9: Flags

Pass as re.findall(pattern, text, flag), or put the short form at the start of the pattern.

Flag	Short form	What it changes
re.IGNORECASE (re.I)	(?i)	ignore upper and lower case: refund matches REFUND
re.MULTILINE (re.M)	(?m)	^ and $ match at the start and end of every line
re.DOTALL (re.S)	(?s)	the dot also matches a newline
re.VERBOSE (re.X)	(?x)	allow spaces and # comments inside the pattern, for readability
re.ASCII (re.A)	(?a)	\w, \d, \s and \b match ASCII only (Unicode is the default)
re.UNICODE (re.U)	(?u)	Unicode matching: already the default for text in Python 3
Combining		re.I | re.S turns on both

EXAMPLE

Flags in Code

The same pattern three times: no flag, a flag, and the inline form.

```
import re

t = "Refund REFUND refund\nOrder"

print(re.findall(r'refund', t))                    # case matters
print(re.findall(r'refund', t, re.IGNORECASE))     # ignore case
print(re.findall(r'(?i)refund', t))                # same, inline flag
print(re.findall(r'refund.order', t))              # . does not match newline
print(re.findall(r'refund.order', t, re.DOTALL | re.IGNORECASE))
# >> ['refund']
#    ['Refund', 'REFUND', 'refund']
#    ['Refund', 'REFUND', 'refund']
#    []
#    ['refund\nOrder']
```

WHY, WHAT, LIMITS

Before We Start: The re Functions

Why it matters, what it is, and where it stops working.

	The re Functions
Why it matters	Knowing the pattern is half the job. Choosing the right function decides whether you get the first match, every match, a replacement or a split
What it is	The re module offers one function per job. search, match and fullmatch return a match object or None. findall returns strings. sub replaces. split cuts. compile prepares a pattern for reuse
Limitations	match only checks the START of the text, which surprises many beginners. findall changes what it returns when the pattern has groups. Always check for None before calling .group()

REGEX RULES

Rules 10: The re Functions

Each takes a pattern and a text.

Function	What it does	Returns
re.search(p, s)	first match anywhere in s	match object, or None
re.match(p, s)	match only at the start of s	match object, or None
re.fullmatch(p, s)	the whole of s must match	match object, or None
re.findall(p, s)	every non-overlapping match	list of strings (or of groups)
re.finditer(p, s)	every match, with its position	iterator of match objects
re.sub(p, new, s)	replace every match with new	new string
re.subn(p, new, s)	same, and count the replacements	(string, count)
re.split(p, s)	cut s wherever p matches	list of strings
re.compile(p)	prepare a pattern once for reuse	pattern object
re.escape(s)	make plain text safe to use in a pattern	escaped string

REGEX RULES

Rules 11: The Match Object

What search, match, fullmatch and finditer give back.

Method	What it gives	Example
m.group() m.group(0)	the whole matched text	'0300-1234567'
m.group(1)	what group 1 captured	'0300'
m.group('name')	what a named group captured	m.group('code')
m.groups()	a tuple of all captured groups	('0300', '1234567')
m.groupdict()	named groups as a dictionary	{'code': '0300', ...}
m.start() m.end() m.span()	where the match begins and ends	(5, 17)
if m:	a match object is true, None is false	check before using it

EXAMPLE

Searching with re

One text, five functions. match returns None because the text does not start with a digit.

```
import re

s = "order 482 and order 517"

print(re.search(r'\d+', s).group())       # first match anywhere
print(re.match(r'\d+', s))                # must match at the START
print(re.fullmatch(r'order \d+', 'order 482'))  # must match ALL of it
print(re.findall(r'\d+', s))              # every match, as strings
print([m.start() for m in re.finditer(r'\d+', s)])  # positions
# >> 482
#    None
#    <re.Match object; span=(0, 9), match='order 482'>
#    ['482', '517']
#    [6, 20]
```

EXAMPLE

Replacing and Splitting with re

sub, subn, split, escape and compile on the same text.

```
import re

s = "order 482 and order 517"

print(re.sub(r'\d+', 'N', s))            # replace
print(re.subn(r'\d+', 'N', s))           # replace and count
print(re.split(r'\s+and\s+', s))         # split on a pattern
print(re.escape('a.b*c'))                # make text literal
pat = re.compile(r'\d+')                 # compile once, reuse
print(pat.findall(s))
# >> order N and order N
#    ('order N and order N', 2)
#    ['order 482', 'order 517']
#    a\.b\*c
#    ['482', '517']
```

WHY, WHAT, LIMITS

Before We Start: Regex Recipes for NLP

Why it matters, what it is, and where it stops working.

	Regex Recipes for NLP
Why it matters	Emails, phone numbers, links, hashtags, dates and order numbers recur in almost every support, social media and form dataset. Ready patterns save time and give a starting point
What it is	A recipe is a pattern that already works for a common shape. Start from one, test it on 20 real messages, then fix what it misses
Limitations	No recipe is complete. Real emails, phone numbers and dates come in many more formats than any short pattern allows. Treat every recipe as a first draft, not a validator

REGEX RULES

Rules 12: Recipes

Starting points, all tested on the sample text on the next slide.

For	Pattern	Notes
Email	[\w.+-]+@[\w-]+\.[\w.]+	good enough to find, not to validate
Pakistani mobile	(?:\+92|0)[\s-]?3\d{2}[\s-]?\d{7}	+92 300 1234567 and 0300-1234567
Link	https?://\S+	stops at the first space
Hashtag	#\w+	letters, digits, underscore
Mention	(?<!\w)@\w+	lookbehind skips the @ in emails
Date dd/mm/yyyy	\d{2}/\d{2}/\d{4}	does not check that the date exists
Repeated letters	(.)\1{2,}	soooo; replace with \1
Slot placeholder	\{\{.*?\}\}	our {{Order Number}} placeholders

EXAMPLE

Recipes in Code

Each recipe applied to one text.

```
import re

t = ("Mail ali@shop.pk or call +92 300 1234567 / 0300-1234567. "
     "See https://shop.pk/track?id=77 #sale @daraz on 05/03/2026")

print(re.findall(r'[\w.+-]+@[\w-]+\.[\w.]+', t))
print(re.findall(r'(?:\+92|0)[\s-]?3\d{2}[\s-]?\d{7}', t))
print(re.findall(r'https?://\S+', t))
print(re.findall(r'#\w+', t), re.findall(r'(?<!\w)@\w+', t))
print(re.findall(r'\d{2}/\d{2}/\d{4}', t))
# >> ['ali@shop.pk']
#    ['+92 300 1234567', '0300-1234567']
#    ['https://shop.pk/track?id=77']
#    ['#sale'] ['@daraz']
#    ['05/03/2026']
```

REGEX LIMITS

Where Regex Fails

Every regex user meets these. Know them before you rely on a pattern.

Limitation	Why it happens	What to do instead
Matches shape, not meaning	a pattern has no idea what the words mean	use a classifier or an NLP model for meaning
Too loose or too strict	an email pattern accepts a junk address or rejects a real one	test on real data; add checks outside the regex
Misses other formats	03001234567 slips past a pattern that wants a dash	list the formats you expect, or normalise first
Nesting	regex cannot count matching pairs, so nested tags break it	use an HTML parser such as BeautifulSoup
Language assumptions	[a-z] and \b assume English; Urdu is lost	use \w or Unicode ranges; test on Urdu text
Hard to read and maintain	a long pattern is a puzzle for the next person	use re.VERBOSE, names, and comments
Can be slow	bad patterns with nested repeats can take very long on some text	keep patterns simple and anchored

EXAMPLE

Limits in Code

Four patterns that look fine and fail. The last two lines show the Urdu problem.

```
import re

print(re.findall(r'[\w.]+@[\w.]+\.\w+', 'ok@shop.pk, no..way@x..y.com'))  # accepts junk
print(re.findall(r'\b\d{4}-\d{7}\b', '03001234567'))             # misses other formats
print(re.findall(r'<b>(.*?)</b>', '<b>a <b>nested</b></b>'))    # nesting breaks it
print(re.findall(r'\w+', 'مجھے refund چاہیے'))                  # Unicode: works
print(re.findall(r'[a-z]+', 'مجھے refund چاہیے'))               # a-z: Urdu lost
# >> ['ok@shop.pk', 'no..way@x..y.com']
#    []
#    ['a <b>nested']
#    ['مجھے', 'refund', 'چاہیے']
#    ['refund']
```

PRACTICE

Practice

Write one regex that finds Pakistani mobile numbers in all of these forms: 0300-1234567, 03001234567, +92 300 1234567 and +923001234567. Test it on all four. Then say what it would still miss.

ANSWER

Practice: Answer

```
# One answer:  (?:\+92|0)[\s-]?3\d{2}[\s-]?\d{7}
#
# (?:\+92|0)   the prefix: +92 or a leading 0 (non-capturing group)
# [\s-]?       an optional space or dash
# 3\d{2}       the operator code: 3 and two more digits
# [\s-]?       an optional space or dash
# \d{7}        the seven subscriber digits
#
# Misses: 0092 as a prefix, spaces in other places, landline
# numbers, and it also matches digits inside a longer number.
# Fix the last with lookaround: (?<!\d)  ...  (?!\d)
```

EXAMPLE

Cleaning Functions

Each function does one job with a regular expression. Together they remove links, HTML, numbers, stretched letters and symbols.

```
import re, string, unicodedata


def to_lower(t):          return t.lower()
def remove_urls(t):       return re.sub(r'https?://\S+|www\.\S+', '', t)
def remove_html(t):       return re.sub(r'<.*?>', '', t)
def remove_numbers(t):    return re.sub(r'\d+', '', t)
def remove_repeats(t):    return re.sub(r'(.)\1{2,}', r'\1', t)
def remove_special(t):    return re.sub(r'[^a-zA-Z\s]', '', t)
def squeeze_spaces(t):    return re.sub(r'\s+', ' ', t).strip()
```

EXAMPLE

Running the Steps in Order

We chain the functions and print the text after each step, so we can see exactly what each one removed.

```
# uses the functions from the previous slide
raw = "OMG!!! Refund for #48213 😍😍 soooo slow www.shop.pk <b>NOW</b>"

t = raw
for step in (to_lower, remove_urls, remove_html, remove_numbers,
             remove_repeats, remove_special, squeeze_spaces):
    t = step(t)
    print(f"{step.__name__:16s}{t}")
# >> to_lower         omg!!! refund for #48213 😍😍 soooo slow www.shop.pk <b>now</b>
#    remove_urls      omg!!! refund for #48213 😍😍 soooo slow  <b>now</b>
#    remove_html      omg!!! refund for #48213 😍😍 soooo slow  now
#    remove_numbers   omg!!! refund for #  😍😍 soooo slow  now
#    remove_repeats   omg! refund for #  😍😍 so slow  now
#    remove_special   omg refund for   so slow  now
#    squeeze_spaces   omg refund for so slow now
```

QUICK CHECK

Quick Check

Our cleaning removed everything except letters and spaces. Run it on "mail help@shop.pk" and "call 0300-1234567". What is left, and is that what you wanted? What would you change?

ANSWER

Quick Check: Answer

```
# 'help@shop.pk' becomes 'helpshoppk': one nonsense word.
# '0300-1234567' disappears completely.
#
# If the email or phone number matters, extract it FIRST (Hour 3,
# named entities), or replace it with a placeholder like ' email '
# before the symbols are stripped. The order of steps matters.
```

EXAMPLE

Emojis and Accents, and a Warning for Urdu

Two more cleaning steps. Be careful with the second one: it assumes the text is English.

```
import re, unicodedata

emoji = re.compile("[\U0001F600-\U0001F64F\U0001F300-\U0001F5FF"
                   "\U0001F680-\U0001F6FF\U0001F1E0-\U0001F1FF]+")

def remove_emojis(t):    return emoji.sub('', t)
def remove_accents(t):
    return unicodedata.normalize('NFKD', t).encode('ascii', 'ignore').decode()

print(remove_emojis("great service 😍😍 thanks"))
print(remove_accents("café at the Nestlé outlet"))
print(remove_accents("مجھے ریفنڈ چاہیے"))      # Urdu: every letter is deleted
# >> great service  thanks
#    cafe at the Nestle outlet
#
```

CONCEPT

What We Left Out on Purpose

- Spelling correction (for example TextBlob's correct()) can repair "recieve", but it also "fixes" names, brands and Roman Urdu words into wrong English words. Test it before trusting it
- Our Bitext data has real typos ("refnd", "rtying"). We leave them: the model can still learn them from the other words in the message
- Removing mentions and hashtags ("@user #AI") matters for social media data, not for support chat
- Rule of thumb: clean only what hurts the task, and look at the output after each step

EXAMPLE

Cleaning Real Support Messages

The same idea on the real data. One extra rule: every {{slot}} placeholder becomes the single word slot, so all order numbers look alike.

```
import pandas as pd, re

df = pd.read_csv('bitext.csv')

def clean(t):
    t = re.sub(r'\{\{.*?\}\}', ' slot ', t.lower())
    return re.sub(r'\s+', ' ', re.sub(r'[^a-z\s]', ' ', t)).strip()

for m in df['instruction'].sample(2, random_state=4):
    print(m)
    print('  ->', clean(m))
# >> open {{Account Category}} account
#      -> open slot account
#    why I cannot set up the shipping address valid
#      -> why i cannot set up the shipping address valid
```

15-Minute Break

Stretch, grab a coffee. We'll resume shortly.

Hour 2

Preprocessing: Making Text Ready for Models

WHY, WHAT, LIMITS

Before We Start: Tokenization

Why it matters, what it is, and where it stops working.

	Tokenization
Why it matters	A model cannot read a whole message at once. It needs separate units to count, look up and compare
What it is	Tokenization splits text into tokens: words, sentences, subwords or characters
Limitations	Simple splitting mishandles punctuation, contractions and languages without spaces. Word tokens meet unknown words, and subword tokens are harder to read

CONCEPT

Tokenization: Splitting Text Into Units

- A token is the unit a model reads. Tokenization breaks a stream of text into tokens
- Word tokens: "My parcel is late" -> My, parcel, is, late. Sentence tokens: split a paragraph into sentences
- Subword tokens: rarer words are split into pieces. This is what LLMs use, so a typo or a new word can still be represented
- Character tokens: every letter is a token. Sequences get very long and each token carries little meaning

EXAMPLE

Tokenizing With NLTK

word_tokenize splits off punctuation as separate tokens. sent_tokenize splits sentences. Plain split() is the simple but crude alternative.

```
from nltk.tokenize import word_tokenize, sent_tokenize

text = "My parcel is late. Refund me!"

print(word_tokenize(text))
print(sent_tokenize(text))
print(text.split())          # split() leaves "late." and "me!" glued
# >> ['My', 'parcel', 'is', 'late', '.', 'Refund', 'me', '!']
#    ['My parcel is late.', 'Refund me!']
#    ['My', 'parcel', 'is', 'late.', 'Refund', 'me!']
```

QUICK CHECK

Quick Check

Why is word_tokenize better than split() for the text above? Give one situation where the difference would hurt a model.

ANSWER

Quick Check: Answer

```
# split() leaves punctuation glued to the word: 'late.' and 'me!'
# are different tokens from 'late' and 'me'. The same word then has
# several entries in the vocabulary ('refund', 'refund!', 'refund.').
# Counts get split across them and the model sees less evidence
# for each one.
```

WHY, WHAT, LIMITS

Before We Start: Stop Words

Why it matters, what it is, and where it stops working.

	Stop Words
Why it matters	Words like 'the', 'is' and 'my' appear in almost every message, so they tell us little. Removing them shrinks the text and puts weight on the content words
What it is	Stop words are a list of very common words that a pipeline drops before counting
Limitations	The standard list includes 'not' and 'no', so removal can flip the meaning. Modern models keep stop words because grammar and negation matter

CONCEPT

Stop Words: Very Common Words

- Stop words are words like "the", "is", "a", "my" that appear everywhere and carry little meaning on their own
- Removing them makes the text shorter and focuses on the content words. It helps search and keyword extraction
- The danger: the standard English list also contains "not" and "no". Removing them can turn "not received" into "received"
- Modern models usually keep stop words, because they carry grammar and negation

EXAMPLE

Stop Word Removal

NLTK ships a list of English stop words. We keep every token that is not on the list.

```
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize

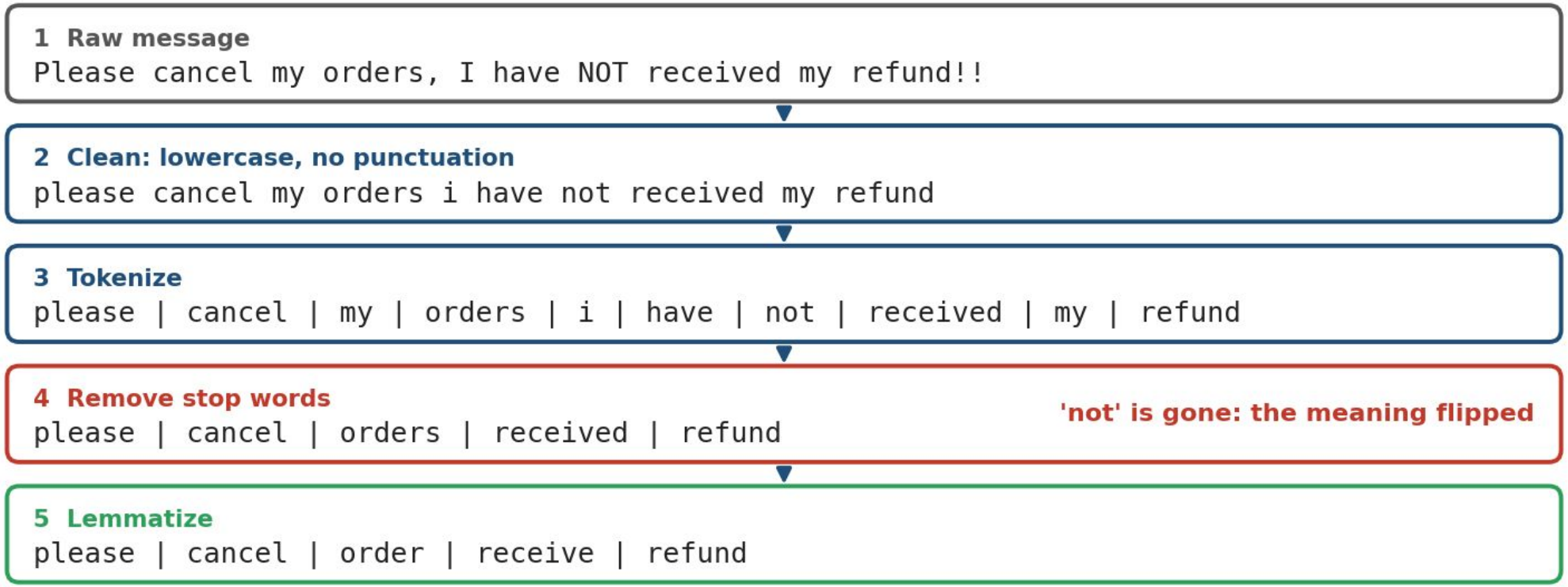
stop = set(stopwords.words('english'))
msg = "i have not received my refund and it is not working"

kept = [w for w in word_tokenize(msg) if w not in stop]
print(kept)
print('not' in stop, 'no' in stop, 'my' in stop)
# >> ['received', 'refund', 'working']
#    True True True
```

EXAMPLE

Our Message Through the Steps

One message, run through cleaning, tokenizing, stop-word removal and lemmatizing. Note what step 4 does to the meaning.



WHY, WHAT, LIMITS

Before We Start: Stemming

Why it matters, what it is, and where it stops working.

	Stemming
Why it matters	'Cancel', 'cancelled' and 'cancelling' mean the same for most tasks. Merging them gives the model more evidence for one feature instead of three weak ones
What it is	Stemming chops word endings with fixed rules to reach a shared root
Limitations	The stem is often not a real word ('delivery' becomes 'deliveri'), it can join unrelated words, and it misses irregular forms such as 'better'

CONCEPT

Stemming: Chop Off the Ending

- Stemming cuts prefixes and suffixes with fixed rules to reach a shared root: "cancelling" and "cancelled" both become "cancel"
- It is fast and needs no dictionary, but the stem is not always a real word: "delivery" becomes "deliveri"
- It also cannot handle irregular words: "better" stays "better"
- The Porter stemmer is the classic English one

EXAMPLE

Porter Stemmer

PorterStemmer().stem() applies the suffix rules to one word at a time.

```
from nltk.stem import PorterStemmer

ps = PorterStemmer()
for w in ["cancelling", "cancelled", "refunds", "delivery", "delivered",
          "happily", "studies", "better"]:
    print(f"{w:11s}", ps.stem(w))
# >> cancelling  cancel
#    cancelled   cancel
#    refunds     refund
#    delivery    deliveri
#    delivered   deliv
#    happily     happili
#    studies     studi
#    better      better
```

WHY, WHAT, LIMITS

Before We Start: Part-of-Speech Tagging

Why it matters, what it is, and where it stops working.

	Part-of-Speech Tagging
Why it matters	The same word can be a noun or a verb ('order' a parcel, an order arrived). The tag tells later steps which meaning to use
What it is	POS tagging labels every word with its grammatical role: noun, verb, adjective, adverb and so on
Limitations	The tagger is itself a trained model and makes mistakes (it called 'Please' a proper noun). It is trained on English and struggles with slang and mixed languages

CONCEPT

Part-of-Speech (POS) Tagging

- POS tagging labels each word with its grammatical role: noun, verb, adjective, adverb, and so on
- NLTK uses the Penn Treebank tag set: NN noun, NNS plural noun, VB verb, VBD past-tense verb, VBG verb ending in -ing, JJ adjective, RB adverb, PRP pronoun, DT determiner
- Why it matters: the same word can play different roles, so the tag helps pick the right meaning. It improves search, entity recognition and lemmatization
- The tagger is itself a trained model, so it makes mistakes

EXAMPLE

POS Tags in NLTK

pos_tag takes a list of tokens and returns (word, tag) pairs. Notice that "Please" at the start was tagged as a proper noun: the tagger is not perfect.

```
from nltk import pos_tag, word_tokenize

msg = "Please cancel my order and send me a refund"
for word, tag in pos_tag(word_tokenize(msg)):
    print(f"{word:8s} {tag}")
# >> Please   NNP
#    cancel   VB
#    my       PRP$
#    order    NN
#    and      CC
#    send     VB
#    me       PRP
#    a        DT
#    refund   NN
```

WHY, WHAT, LIMITS

Before We Start: Lemmatization

Why it matters, what it is, and where it stops working.

	Lemmatization
Why it matters	Reduce 'were', 'is' and 'being' to 'be' and 'parcels' to 'parcel' and the vocabulary shrinks, while every result is still a real, readable word
What it is	Lemmatization looks up each word's dictionary form, using its meaning and part of speech
Limitations	It needs correct POS tags, a dictionary for the language, and more time than stemming. It does not fix typos or slang that the dictionary does not contain

CONCEPT

Lemmatization: Find the Real Base Word

- A lemma is the dictionary form of a word: "running" -> "run", "were" -> "be", "parcels" -> "parcel"
- Unlike stemming, lemmatization uses the word's meaning and part of speech, and always returns a real word
- Three ways to do it. Rule-based: apply suffix rules ("walked" -> "walk"); fails on irregular words. Dictionary-based: look the word up in a lexicon such as WordNet; handles exceptions. Model-based: a trained model decides from context; most flexible, as in spaCy
- The tool needs the POS tag to choose correctly: "saw" is "see" as a verb but "saw" as a noun

EXAMPLE

WordNet Lemmatizer With POS

WordNet uses its own tag codes, so a small function converts the Penn tag to a WordNet tag. Then lemmatize() gets the word and the tag.

```
from nltk import pos_tag, word_tokenize
from nltk.corpus import wordnet
from nltk.stem import WordNetLemmatizer

def wn_pos(tag):
    return {'J': wordnet.ADJ, 'V': wordnet.VERB,
            'R': wordnet.ADV}.get(tag[0], wordnet.NOUN)

lem = WordNetLemmatizer()
text = "The customers were running late"
for w, tag in pos_tag(word_tokenize(text)):
    print(f"{w:10s} {tag:4s} {lem.lemmatize(w, wn_pos(tag))}")
# >> The        DT   The
#    customers  NNS  customer
#    were       VBD  be
#    running    VBG  run
#    late       RB   late
```

CONCEPT

Stemming or Lemmatization?

Same goal, different trade-off.

	Stemming	Lemmatization
Method	Chop the ending with rules	Dictionary and part of speech
Result	Can be a non-word ("deliveri")	Always a real word ("deliver")
Speed	Very fast	Slower, needs POS tags
Irregular words	Misses them ("better")	Handles most ("were" -> "be")
Use when	Speed matters, search engines	Meaning matters, you will read the output

PRACTICE

Practice

Practice. Take any 5 messages from the Bitext data. Run them through clean(), word_tokenize, stop-word removal and the WordNet lemmatizer. For each, write what you lost and what you kept. Did any stop-word removal change the meaning?

15-Minute Break

Stretch, grab a coffee. We'll resume shortly.

Hour 3

Entities, N-grams and Turning Words Into Numbers

WHY, WHAT, LIMITS

Before We Start: Named Entity Recognition

Why it matters, what it is, and where it stops working.

	Named Entity Recognition
Why it matters	Support, banking and logistics all need the facts inside a message: the order number, the city, the amount, the date. NER pulls them out so a program can act on them
What it is	NER finds the important items in text and gives each a type such as PERSON, LOCATION, DATE or MONEY
Limitations	Dictionaries miss anything not listed, rules only find fixed shapes, and trained models inherit the bias of their training data: spaCy called Daraz a person

CONCEPT

Named Entity Recognition (NER)

- NER finds the important items in text and gives each a type: PERSON, ORGANIZATION, LOCATION, DATE, MONEY and so on
- For support: pull out the order number, the product and the delivery city from a free-text complaint
- Context decides the type: "Amazon is expanding rapidly" (organization) versus "The Amazon is the largest rainforest" (place)
- Three approaches: a dictionary of known names, hand-written rules, or a trained model

CONCEPT

Three Ways to Find Entities

Each one trades effort against flexibility.

Method	How it works	Good at	Weak at
Lexicon (dictionary)	Look up every word in a list of known names	Fixed, known names	Anything not in the list
Rule-based	Patterns such as an email or phone format	Structured codes	Names, other formats
Machine learning (spaCy)	A trained model reads the context	Names and context	Needs data, can be wrong

EXAMPLE

NER With a Dictionary

The simplest approach: a lexicon, and a loop that checks each word against it.

```
lexicon = {
    "person":   ["ali", "ahmed", "sara"],
    "city":     ["karachi", "lahore", "islamabad"],
    "company":  ["google", "daraz", "openai"],
}

def lexicon_ner(text):
    return [(w, kind) for w in text.lower().split()
            for kind, names in lexicon.items() if w in names]

print(lexicon_ner("Sara ordered from Daraz and it reached Lahore late"))
print(lexicon_ner("Bilal ordered from Foodpanda"))     # not in the dictionary
# >> [('sara', 'person'), ('daraz', 'company'), ('lahore', 'city')]
#    []
```

EXAMPLE

NER With Rules

Regular expressions suit entities with a fixed shape: emails, phone numbers, order IDs. The second call shows the weakness: the rule only knows one phone format.

```
import re

def rule_ner(text):
    out = []
    for m in re.findall(r'\b[\w.-]+@[\w.-]+\.\w+\b', text):  out.append((m, "EMAIL"))
    for m in re.findall(r'\b\d{4}-\d{7}\b', text):           out.append((m, "PHONE"))
    for m in re.findall(r'#\d{4,}', text):                   out.append((m, "ORDER"))
    return out

msg = "Order #48213 is late. Call 0300-1234567 or mail ali@shop.pk"
print(rule_ner(msg))
print(rule_ner("my number is 03001234567"))        # no dash: the rule misses it
# >> [('ali@shop.pk', 'EMAIL'), ('0300-1234567', 'PHONE'), ('#48213', 'ORDER')]
#    []
```

EXAMPLE

NER With spaCy

spaCy loads a pretrained model and runs a whole pipeline: tokens, tags, lemmas and entities. doc.ents holds the entities it found. The model name is en_core_web_sm.

```
import spacy

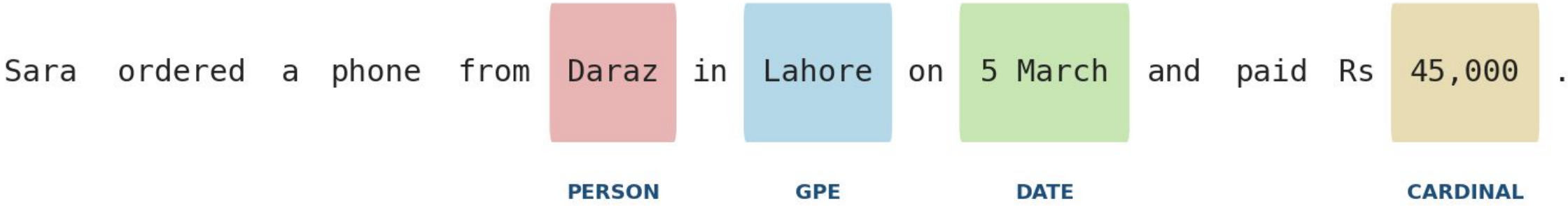
nlp = spacy.load("en_core_web_sm")
doc = nlp("Sara ordered a phone from Daraz in Lahore on 5 March and paid Rs 45,000.")

for ent in doc.ents:
    print(f"{ent.text:12s} {ent.label_}")
# >> Daraz        PERSON
#    Lahore       GPE
#    5 March      DATE
#    45,000       CARDINAL
```

EXAMPLE

What spaCy Found

The same sentence, with the entities highlighted. spaCy found the city, the date and the amount, but called the company Daraz a PERSON and missed Sara.



QUICK CHECK

Quick Check

spaCy tagged "Daraz" as a PERSON and missed "Sara". Why would a model trained on mostly English news text make this mistake, and what are two ways to fix it for a Pakistani support desk?

ANSWER

Quick Check: Answer

```
# The model learned from English news and web text. Daraz is rare
# there, and local first names are less common than Western ones.
#
# Fix 1: add a lexicon or rule on top for names you know:
#        Daraz, Foodpanda, JazzCash, Karachi, Lahore.
# Fix 2: fine-tune the entity model on labelled examples of your
#        own messages. This needs data and labelling time.
```

WHY, WHAT, LIMITS

Before We Start: N-grams

Why it matters, what it is, and where it stops working.

	N-grams
Why it matters	Single words lose order. 'Not working' and 'working' mean opposite things. N-grams keep short runs of words together so a little order survives
What it is	An n-gram is a run of n consecutive words: a unigram is one word, a bigram two, a trigram three
Limitations	The number of features grows very fast (1,199 to 15,361 on our data), most n-grams are rare, and they still only see a short window of context

CONCEPT

N-grams: Keeping a Little Word Order

- An n-gram is a run of n consecutive words. 1 word is a unigram, 2 a bigram, 3 a trigram
- "where is my refund" has the bigrams (where, is), (is, my), (my, refund)
- Bag of words loses order. Bigrams bring a little back: "not working" becomes one feature, not two separate words
- The cost: the number of features grows fast, and most n-grams are rare

EXAMPLE

Generating N-grams

A sliding window of size n over the tokens.

```
def ngrams(text, n):
    tokens = text.split()
    return [tuple(tokens[i:i + n]) for i in range(len(tokens) - n + 1)]

text = "where is my refund"
print(ngrams(text, 1))
print(ngrams(text, 2))
print(ngrams(text, 3))
# >> [('where',), ('is',), ('my',), ('refund',)]
#    [('where', 'is'), ('is', 'my'), ('my', 'refund')]
#    [('where', 'is', 'my'), ('is', 'my', 'refund')]
```

EXAMPLE

N-grams Make the Vocabulary Explode

CountVectorizer builds the n-gram vocabulary for us. min_df=2 drops anything seen in only one message. Compare the feature counts on our 26,872 messages.

```
import pandas as pd
from sklearn.feature_extraction.text import CountVectorizer

df = pd.read_csv('bitext.csv')
msgs = df['instruction'].str.lower()

for rng in [(1, 1), (1, 2), (1, 3)]:
    v = CountVectorizer(ngram_range=rng, min_df=2)
    v.fit(msgs)
    print(rng, len(v.vocabulary_), 'features')
# >> (1, 1) 1199 features
#    (1, 2) 5767 features
#    (1, 3) 15361 features
```

CONCEPT

Vectorization: Words to Numbers

- A model needs numbers, so every text becomes a vector. There are three main ways
- Bag of Words: count how often each vocabulary word appears. Order is ignored
- TF-IDF: like counting, but weigh each word by how rare it is across all texts, so rare, informative words count more
- Word embeddings (Word2Vec, GloVe, and the ones inside LLMs): each word is a short dense vector, where similar words get similar vectors. We meet these on Day 24

WHY, WHAT, LIMITS

Before We Start: Bag of Words

Why it matters, what it is, and where it stops working.

	Bag of Words
Why it matters	Models need numbers. Counting how many times each vocabulary word appears is the simplest way to turn a message into a row of numbers
What it is	Bag of words builds a vocabulary of all distinct words and represents each text as a row of counts, one per word
Limitations	Word order is lost, every word counts equally, the table is huge and mostly zeros, and 'refund' and 'reimbursement' look unrelated

CONCEPT

Bag of Words: Its Two Parts

- Vocabulary: the list of all unique words in the whole dataset. Each word becomes one column (a feature)
- Document representation: each text becomes a row of counts, one count per vocabulary word
- Most counts are zero, so scikit-learn stores the table as a sparse matrix
- Weakness: word order is lost, and every word counts the same, so "my" counts as much as "refund"

WHY, WHAT, LIMITS

Before We Start: TF-IDF

Why it matters, what it is, and where it stops working.

	TF-IDF
Why it matters	In bag of words, 'my' counts as much as 'refund'. TF-IDF gives rare, informative words more weight, which usually makes classifiers more accurate
What it is	TF-IDF multiplies how often a word appears in this text by how rare it is across all texts
Limitations	It still ignores word order and meaning, so synonyms stay unrelated. Its weights depend on the dataset it was fitted on, so it must be fitted on training data only

CONCEPT

TF-IDF: The Idea

- Term Frequency (TF): how often a word appears in this text
- Inverse Document Frequency (IDF): lowers the weight of words that appear in many texts and raises the weight of rare ones
- TF-IDF = TF x IDF. A word that is frequent here but rare everywhere else scores highest
- Example: "refund" is a strong clue to an intent, "my" is not, because almost every message has "my"

EXAMPLE

TF-IDF on Three Tiny Messages

TfidfVectorizer learns the vocabulary and the idf values, then turns each message into a row of weights. Each row is scaled to length 1.

```
from sklearn.feature_extraction.text import TfidfVectorizer

docs = ["where is my refund", "where is my order", "refund refund refund"]
tfidf = TfidfVectorizer()
X = tfidf.fit_transform(docs)
for w, idf in zip(tfidf.get_feature_names_out(), tfidf.idf_):
    print(f"{w:7s} idf = {idf:.2f}")
print(X.toarray().round(2))
# >> is      idf = 1.29
#    my      idf = 1.29
#    order   idf = 1.69
#    refund  idf = 1.29
#    where   idf = 1.29
#    [[0.5  0.5  0.   0.5  0.5 ]
#     [0.46 0.46 0.6  0.   0.46]
#     [0.   0.   0.   1.   0.  ]]
```

CONCEPT

Reading the TF-IDF Output

- "order" has the highest idf (1.69): it appears in only one of the three messages. The other words each appear in two of the three, so they score lower
- In message 3, "refund" is the only word, so its weight is 1.0
- In message 2, "order" gets the largest weight (0.60), because it is the one word that tells message 2 apart
- Each row can now go into any model from Weeks 3 and 4

CONCEPT

What TF-IDF Is Used For

- Document similarity and clustering: group news, papers or support tickets by topic
- Text classification: spam filtering, sentiment analysis, intent detection (tomorrow)
- Keyword extraction: the highest-weighted words of a document are its keywords
- Search: rank documents by how well their TF-IDF vector matches the query

WHY, WHAT, LIMITS

Before We Start: Word Embeddings

Why it matters, what it is, and where it stops working.

	Word Embeddings
Why it matters	Bag of words cannot tell that 'refund' and 'reimbursement' are close. Embeddings put similar words close together, so a model can reuse what it learned about one word for the other
What it is	An embedding is a short list of numbers for each word, learned from how the word is used in text
Limitations	One vector per word cannot capture different meanings of the same word. The vectors carry the biases of their training text, and they need a lot of text to learn well

CONCEPT

Word Embeddings: A First Look

- A word embedding is a list of numbers (say 100) that represents one word, learned from how the word is used
- Words used in similar contexts get similar vectors: "refund" and "reimbursement" end up close together, "refund" and "password" far apart
- This fixes the main weakness of bag of words: it treats "refund" and "reimbursement" as unrelated columns
- Day 24 builds small word vectors from our own messages and shows the neighbours

PRACTICE

Practice

Practice 1. Run CountVectorizer with ngram_range=(1,2) on the Bitext messages and list the 10 most frequent bigrams. Which look useful for telling intents apart, and which are just noise?

PRACTICE

Practice

Practice 2. Write a rule-based extractor for order numbers in the Bitext data. First look at how order numbers appear in 20 messages. Then write one regex and test it. What cases does it miss?

SUMMARY

Key Takeaways

- NLP turns language into something a program can use. The pipeline: collect, clean, preprocess, features, model
- Cleaning is a chain of small functions. Check the result after every step, and remember that order matters
- Tokenize, then decide about stop words, stems or lemmas. Each choice can remove meaning as well as noise
- NER finds names, dates and amounts with a lexicon, rules or a trained model. Each has different failure cases
- Text becomes numbers through bag of words, TF-IDF or embeddings. Tomorrow we use TF-IDF to build a classifier and a chatbot